

INTRODUCTION TO INFO. & COMM. TECHNOLOGY

08-11-2023

Programming Languages

- Two categories of programming languages:
- Low level
- High level
- **Low-level language** → resembles the numeric machine language of the computer
- **High-level languages** → easiest for people to learn
- → closer to the level of human-readability

Low Level Language

■
Low level (machine language)
10100010 11101011

High Level Languages

- Table 1.1 in your book

Source Code

- The statements written by the programmer are called *source code*
- File they are saved in is called the *source file*

Object Code

- After the source code is saved to a file
- Need to translate it to machine language
- Translation:
 - **first phase** → a program called the *preprocessor* reads the source code
 - The preprocessor searches for special lines that begin with the `#` symbol
 - These lines contain commands that cause the preprocessor to modify the source code in some way

Object Code

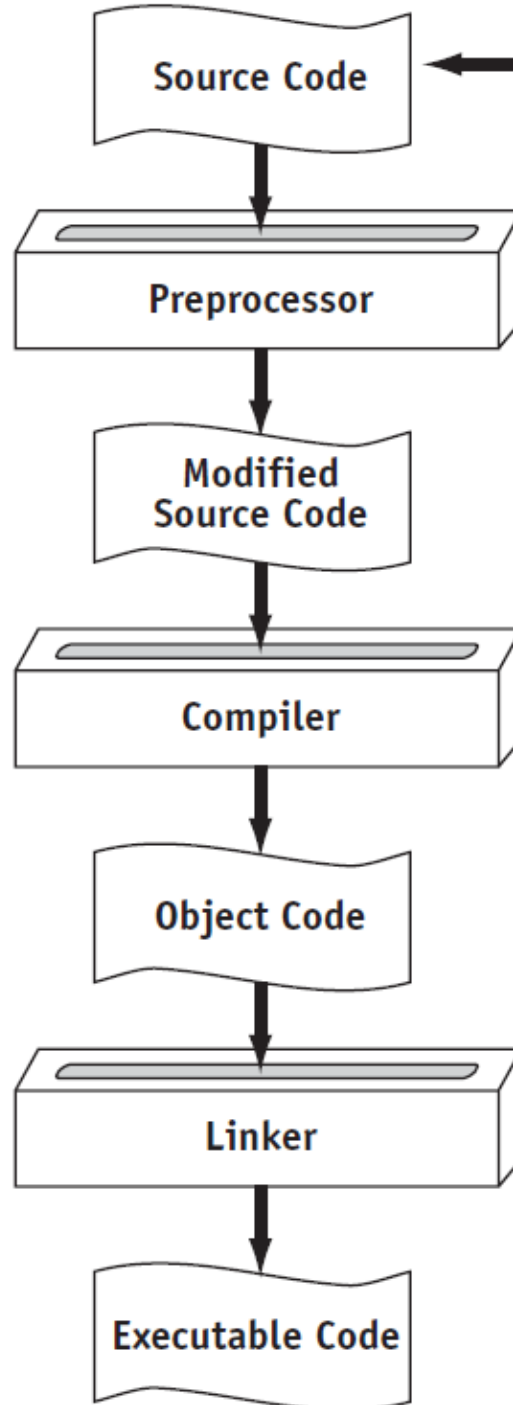
- **Second phase** : the *compiler* uses the preprocessed source code
- Translates each source code instruction into the appropriate machine language instruction
- Finds out ***syntax errors***
- Syntax errors are illegal uses of key words, operators, punctuation, and other language element
- If the program has no syntax errors, the compiler stores the translated machine language instructions, which are called *object code* , in an ***object file***

Object Code

- To have a complete object file
- C++ is conveniently equipped with a library of prewritten code for:
 - displaying messages on the screen
 - reading input from the keyboard
 - routines
 - for mathematical functions e.g. square root
- This collection of code is called the **run-time library**
- **Last phase** of the translation process
- A program called the *linker* combines the object file with the necessary library routines

Executable Code

- Once the linker is done → an *executable file* is created
- The executable file → ready to run on the computer



Source code is entered with a text editor by the programmer.

```
#include <iostream>
using namespace std;

int main()
{
    cout<<"Hello World\n";
    return 0;
}
```

What Is a Program Made of?

- Key words
- Programmer Defined Identifiers
- Operators
- Punctuation
- Syntax

Keywords

- Words that have a special meaning
- Key words may only be used for their intended purpose
- Key words are also known as **reserved words**
- In C++, key words are written in all lowercase

Programmer-Defined Identifiers

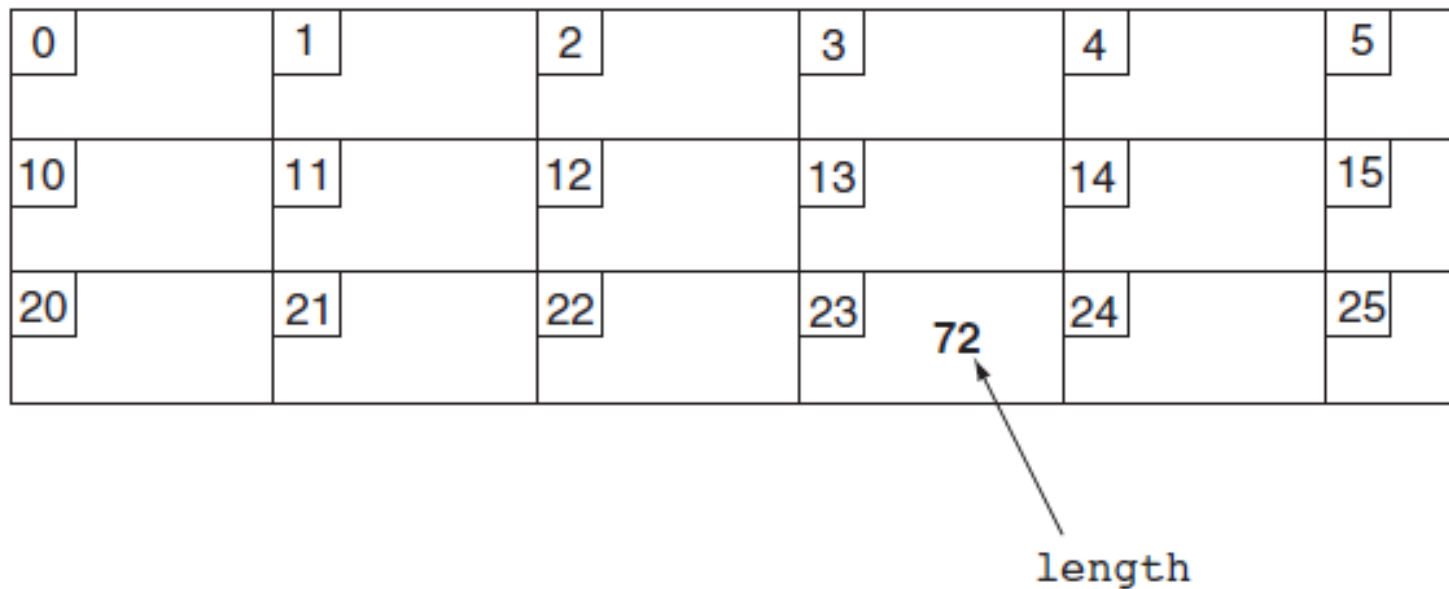
- Words or names defined by the programmer
- These are symbolic names
- Refer to variables or programming routines
- sum, n, num1, i, x, sqr, answer etc.

Variables

- To store information to be
- To label data with a descriptive name
- For better understanding of the programmer and the reader
- A variable → a containers that holds information
- The information stored in variables may change while the program is running

Memory Picture

- Variables are symbolic names that represent locations in the computer's random-access memory (RAM)



Variable Definition

- In programming, there are two general types of data: numbers and characters
- Numbers are used to perform mathematical operations
- Characters are used to print data on the screen or on paper (hard copy)

Numeric Data

- Whole numbers, or **integers**:
 - 5
 - 7
 - -129
 - 32154
- Real, or **floating-point** numbers:
 - 3.14159
 - 6.7
 - 1.0002

Operators

- Operators perform operations on one or more operands
- An operand is usually a piece of data, like a number
- `radius = 10;`
- The `equal to` symbol is called the *assignment operator*
- Operators perform operations on data
- The data that operators work with are called *operands*
- The assignment operator has two operands
- `C++` requires the name of the variable receiving the assignment to appear on the left side of the operator

Arithmetic Operators

- Generally, there are three types of operators:
- *unary* , *binary* , and *ternary*
- These terms reflect the number of operands an operator requires
- **Unary** operators only require a single operand
- For example, consider the following expression: -5
- **The minus sign, when used this way, is called the *negation operator***

Arithmetic Operators

- **Binary operators** work with two operands
- The assignment operator, arithmetic operators
- **Ternary operators** require three operands
- C++ only has one ternary operator (TBDL)

Arithmetic Operators

Operator	Meaning	Type
+	Addition	Binary
-	Subtraction	Binary
*	Multiplication	Binary
/	Division	Binary
%	Modulus	Binary

Punctuation

- Punctuation characters that mark the beginning or ending of a statement
- May separate items in a list

Syntax

- Rules that must be followed when constructing a program
- Tells us how key words and operators may be used
- Syntax tells us where punctuation symbols must appear

Special characters in C++

// **Double slash** → Marks the beginning of a comment

Pound sign → (hash sign) Marks the beginning of a preprocessor directive

#include <iostream>

<> **Angle brackets** → Encloses a filename when used with the #include directive

() **Parentheses** → Used in naming a function, as in int main()

Special characters in C++

{ } **Braces** → Encloses a group of statements, such as the contents of a function

“ “ **quotation marks** → Encloses a string of characters, such as a message that is to be printed on the screen

; **semicolon** → Marks the end of a complete programming statement

Why Variables?

- Most of the time we don't know the actual numbers or other data that the user will enter
- We assign the input data to a program variable
- A variable is called a variable because it can vary!
- A quantity that can change value during the execution of a program
- Whenever we need to refer to that data in a program statement, we simply refer to its variable name

Named Constant

- The value 3.14 cannot be changed during program execution → it is referred to as a program constant
- Sometimes you may want to give a constant a name → When you do, it is called a **named constant**
- Your program may use the value of **pi** at several places after its initialization

```
pi = 3.14;
```

Rules for Variable Names

- All variable names must be one word → any sequence of characters of text and can consist of one or more characters
- Underscores are allowed and hyphens may be allowed, but **spaces are never allowed**
- **Miles_traveled** is fine but **Miles traveled** is not correct
- **Cost-per-gallon** is fine but **cost per gallon** is not

Rules for Variable Names

- Variable names can be long (like more than 200chars)
- Avoid too long or too complicated names
- Variable names can also be short; a single character is an acceptable variable name
- At times one letter rarely indicates what the variable represents, it is best to reserve this type of variable name for specific circumstances
- A, x, or N are all acceptable variable names

Rules for Variable Names

- Digits (0 to 9) are not valid variable names
- Special characters such as # or \$ are not valid characters to be included in variable names
- Many programmers use uppercase letters to distinguish one word from another within a single name. This is sometimes referred to as **CamelBack** (or CamelCase) notation
- **MilesTraveled** works as well as Miles_Traveled and may be easier to type
- **CostPerGallon** works as well as Cost_Per_Gallon and may be easier to type

Rules for Variable Names

- Most languages allow numbers to be part of variable names, but variable names may **never start with a number**
- **TaxRate_1** is fine but **1_TaxRate** is not
- **Destination2** is fine but **2Destination** is not

Assignment Statement

```
ItemPrice = 99;
```

Assigns the value 99 to the variable ItemPrice

```
NoOfItems = 10;
```

```
Bill = ItemPrice * NoOfItems;
```

Multiplies ItemPrice by NoOfItems and then assigns the resulting value of the expression on the **RIGHT** of the equals sign to the variable (Bill) on the **LEFT** side

Assigning & Reassigning

NumberX = 45;

NumberX = 97;

- NumberX first contains the value of 45
- Then value of 45 is replaced by 97
- NumberX will keep the value of 97 until some other statement reassigns it a new value
- In terms of storage locations, the value currently stored in location NumberX (45) is erased and the new value (97) is stored in its place

Assigning & Reassigning

Counter = Counter + 1;

- First the right side is evaluated, and 1 is added to the current value of the variable Counter
- Then this new value is assigned to the variable on the left

OR

- Add one to the previous value of Counter
- If the value was 15 prior to execution of this statement, its value would be 16 afterward

Operations on Data

Table 1.1 Arithmetic Operators

Operator	Computer Symbol	Example
Addition	+	$2 + 3 = 5$
Subtraction	-	$7 - 3 = 4$
Multiplication	*	$5 * 4 = 20$
Division	/	$12 / 3 = 4$
Exponentiation	^	$2 ^ 3 = 8$
Modulus	%	$14 \% 4 = 2$

Writing Expressions in Programs

$$C = \frac{5(F - 32)}{9}$$

In a programming language, the formula is written as:

```
C = 5 * (F - 32) / 9
```

C is the variable for number of degrees in Celsius, and **F** is the variable for the number of degrees in Fahrenheit

Operator Precedence

- The order in which arithmetic operations is performed (i.e., their *hierarchy*) is as follows:
 1. Operations in parentheses (from the inside out, if there are parentheses within parentheses)
 2. Exponentiations
 3. Multiplications, divisions, and modulus (from left to right if there is more than one in one statement)
 4. Additions and subtractions (from left to right if there is more than one in one statement)

Operator Precedence

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they are evaluated left to right.
*, /, or %	Multiplication Division Modulus	Evaluated second. If there are several, they are evaluated left to right.
+ or -	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Refer to Appendix C in your book to understand this better

Data Types

- The computer allocates a certain amount of space in its memory for data
- Some data requires more space than other data
- E.g. a single alphanumeric character (the letters A through Z, a through z, and the digits 0 through 9) requires eight bits of space
- A floating point number (such as 3.7, -43.9825, or 126,854,927.38726) requires much more space in memory
- The type of data determines how much memory is needed

Data Types

- When we write programs, we must define the **data type** of the variables
- Computer languages make use of two fundamental data types: **numeric data** and **character string** (or **alphanumeric**) **data**
- Numeric data can be further divided into two major types : Integer and floating
- Data that consists of text is divided into two types:
- Character data → a single alphanumeric character
- String data → consists of as much text as desired
- Boolean data → TBDL

Downloading Dev C++

- Contact Mr. Osama Tariq in SCIT for support regarding the installation of Dev C++ in your systems so you will be able to practice at home.

Osama Tariq osama.tariq@bnu.edu.pk

Ref Chapters

- Book 3
- Chp 1,2